SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 Total Marks:50

Annexure A

Scenario based Questions

Code of Conduct:

I **G. Puneeth (192524221)** certify that this submission is my original work and that

I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

G. Puneeth

**Question 1 – CPU Scheduling in a Real-Time Ticket Booking System**

**1. Most Suitable Scheduling Algorithm**

**Priority Scheduling** is the most suitable algorithm because payment transactions are critical and should be processed before seat checking and ticket printing.

**2. Justification**

- Executes high-priority payment requests immediately.
- Reduces waiting time for important tasks.
- Improves CPU utilization by processing urgent jobs first.
- Enhances customer satisfaction by completing transactions quickly.

**3. Sample Execution Order**

| **Process** | **Priority** | **Execution Order** |
|---|---|---|
| P1 – Payment Gateway | High | 1st |
| P2 – Seat Availability | Medium | 2nd |
| P3 – Ticket Confirmation | Low | 3rd |

**4. Drawback**

Priority Scheduling may cause **starvation**, where low-priority processes (ticket confirmation) wait indefinitely if high-priority requests continue arriving.

**Question 2 – Deadlock in a Banking Transaction Server**

**1. Resource Allocation Graph (Text Form)**

T1 holds A → requests B

T2 holds B → requests C

T3 holds C → requests A

Or

A → T1 → B

B → T2 → C

C → T3 → A

**2. Has a Deadlock Occurred?**

**Yes.**

Because there is a circular wait:

- T1 waits for B
- T2 waits for C
- T3 waits for A

No process can continue.

**3. Recommended Technique**

**Deadlock Prevention**

Enforce a fixed order of acquiring resources (A → B → C). This removes the circular wait condition and prevents deadlocks.

**4. Redesign**

Require every transaction to request resources in the same sequence:

Account Database (A)

↓

Customer Info Database (B)

↓

Transaction Logs (C)

Since every process follows the same order, deadlock cannot occu

**Question 3 – Mobile OS App Switching Lag**

**1. Why CPU Shows High Idle Time**

- Many apps are waiting for I/O operations.
- Frequent context switching wastes CPU time.
- Background applications consume scheduling time without doing useful work.

**2. Is 20 ms Quantum Helping?**

No.

A 20 ms time quantum with 20 active applications causes excessive context switching, increasing delay during app switching.

**3. Better Scheduling Method**

Use:

- **Priority Scheduling** for foreground applications.
- Increase Round Robin quantum to **40–50 ms**.

This gives active apps more CPU time while reducing context switching.

**4. Benefits**

- Faster app switching.
- Reduced context switching overhead.
- Better CPU utilization.
- Improved throughput and smoother user experience.

**Question 4 – Deadlock Risk in an Operating System Update**

**1. Can This Lead to Deadlock?**

**Yes.**

The resource allocation forms a cycle.

**2. Circular Wait Condition**

M1 holds R1 → waits for R2

M2 holds R2 → waits for R3

M3 holds R3 → waits for R1

Cycle:

M1 → R2 → M2 → R3 → M3 → R1 → M1

Hence, a deadlock can occur.

**3. Banker's Algorithm Style Allocation**

Before granting a resource:

- Check if the system remains in a **safe state**.
- Allocate resources only if all modules can eventually complete.
- Delay unsafe requests until resources become available.

**4. Improve CPU Utilization**

Assign priorities:

1. M1 – Copy update files
2. M2 – Install drivers
3. M3 – Update registry
4. M4 – Validate system integrity

This reduces waiting time and keeps the CPU busy with executable tasks.

**Question 5 – Cloud Server CPU Utilization Analysis**

Given:

- **p = 0.8** (I/O wait probability)
- **n = 7** VMs

Formula:

$$U = 1 - p^n$$

**1. CPU Utilization**

$$U = 1 - (0.8)^7$$
$$0.8^7 = 0.2097$$
$$U = 1 - 0.2097 = 0.7903$$

**CPU Utilization ≈ 79%**

**2. Interpretation**

A utilization of about **79%** indicates the server is **well utilized**. There is still some idle time due to I/O waits, but overall CPU usage is efficient.

**3. Effect of Changing Number of VMs**

- **Increase VMs:** Better CPU utilization and higher throughput (up to an optimal limit), but too many VMs can cause contention and reduce performance.
- **Decrease VMs:** Lower contention but increased CPU idle time and reduced throughput.

**4. Two OS-Level Strategies**

- Use **multiprogramming** to schedule another VM while one waits for I/O.
- Implement **efficient CPU scheduling** (such as Multilevel Feedback Queue or optimized Round Robin) to minimize idle time and improve throughput.

**Question 6 – CPU Scheduling Scenario**

**1. Analyze the Root Cause**

The organization experiences performance issues because:

- Many processes compete for CPU resources.
- Poor scheduling causes long waiting times.
- High context switching reduces CPU efficiency.
- Resource contention lowers throughput.

**2. Most Appropriate OS Technique**

**Multilevel Feedback Queue (MLFQ) Scheduling**

MLFQ dynamically adjusts process priorities based on CPU usage and waiting time.

**3. Justification**

- Improves CPU utilization by keeping the CPU busy.
- Provides quick response for short and interactive jobs.
- Prevents starvation through priority adjustment.
- Increases throughput and ensures fairness.

**4. Improved System Configuration**

Execution Order:

Interactive Processes

↓

Short CPU-bound Processes

↓

Long CPU-bound Processes

↓

Background Processes

Processes move between queues according to their execution behavior.

**5. Limitation**

MLFQ is more complex to implement and requires careful tuning of queue priorities and time quantum.

**Question 7 – Process Management in an E-Commerce Company**

**1. Analyze the Processes**

The OS manages several independent processes:

- User Authentication
- Product Search
- Shopping Cart
- Payment Processing
- Inventory Management
- Shipment Tracking

Each process executes concurrently and communicates with others.

**2. Program vs Process**

| **Program** | **Process** |
|---|---|
| Static set of instructions | Program in execution |
| Stored on disk | Loaded into memory |
| Example: Shopping App | Customer placing an order |

**3. Process Control Block (PCB)**

PCB stores:

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Scheduling Information
- Memory Information
- I/O Status

The OS uses PCB to manage and switch between processes.

**4. Payment Transaction Process States**

New

↓

Ready

↓

Running

↓

Waiting (Payment Verification)

↓

Ready

↓

Running

↓

Terminated

**5. Impact of Context Switching**

- Increases CPU overhead.
- Reduces throughput.
- Causes slower response during festive sales.

**6. Consequences of Excessive Process Creation**

- High memory usage.
- Increased CPU overhead.
- Frequent context switching.
- Reduced system performance.

**7. Efficient Process Hierarchy**

E-Commerce Server

│

├── Authentication

├── Product Search

├── Shopping Cart

├── Payment

├── Inventory

└── Shipment Tracking

**8. Need for Multiprogramming**

- Keeps CPU busy while other processes wait for I/O.
- Improves throughput.
- Increases resource utilization.

**9. Recommendations**

- Use multithreading.
- Apply load balancing.
- Use efficient CPU scheduling.
- Increase server resources during peak sales.

**10. Process Lifecycle Model**

New

↓

Ready

↓

Running

↓

Waiting

↓

Ready

↓

Running

↓

Terminated

**Question 8 – Process Scheduling in a Smart Hospital**

**1. Scheduling Challenges**

- Emergency services require immediate CPU access.
- Routine tasks compete for CPU.
- Delays affect patient care.

**2. Comparison**

| **Algorithm** | **Advantages** | **Disadvantages** |
|---|---|---|
| FCFS | Simple | Long waiting time |
| SJF | Minimum average waiting time | Needs burst time prediction |
| Priority | Best for emergencies | Starvation possible |
| Round Robin | Fair CPU sharing | High context switching |

**3. Impact on Emergency Care**

Priority Scheduling allows emergency tasks to execute immediately, reducing treatment delay.

**4. Best Scheduling Algorithm**

**Priority Scheduling** (with Aging)

Emergency processes receive highest priority while aging prevents starvation.

**5. Starvation and Remedy**

Starvation may occur for low-priority tasks.

**Solution:** Aging gradually increases waiting process priority.

**6. Suitable Time Quantum (if RR is used)**

**30–40 ms** provides a balance between response time and context-switch overhead.

**7. Mass Casualty Incident**

- Emergency processes dominate CPU.
- Routine tasks are delayed.
- Aging ensures routine tasks eventually execute.

**8. Justification**

Priority Scheduling provides:

- Fast response time.
- Better CPU utilization.
- Improved patient safety.

**9. Improvements**

- Aging technique.
- Dynamic priorities.
- Separate emergency queue.
- Load balancing.

**10. Scheduling Framework**

Emergency Queue (Highest)

↓

Critical Services

↓

Routine Services

↓

Background Tasks

**Question 9 – Inter-Process Communication (IPC) in a Smart City**

**1. Cooperating Processes**

- Traffic Signals
- CCTV Cameras
- Weather Stations
- Public Transport
- Emergency Vehicle Tracking

**2. Communication Requirements**

Processes exchange:

- Traffic density
- Weather updates
- Emergency vehicle location
- Signal timing
- Public transport status

**3. Shared Memory vs Message Passing**

| **Shared Memory** | **Message Passing** |
|---|---|
| Very fast | More secure |
| Needs synchronization | Easier communication |
| Suitable for same system | Suitable for distributed systems |

**4. Best IPC Mechanism**

**Message Passing**

Because smart city components are distributed across different locations.

**5. Impact of Communication Failure**

- Traffic congestion.
- Delayed emergency response.
- Increased accident risk.

- Reduced public safety.

**6. Fault-Tolerant IPC Architecture**

Subsystem

↓

Message Queue

↓

Central Controller

↓

Backup Controller

**7. Future Challenges**

- Increased traffic data.
- Network congestion.
- Scalability.
- Security threats.

**8. Improve Synchronization**

- Semaphores
- Mutex Locks
- Monitors
- Message Queues

**9. Secure Communication Framework**

- Encryption
- Authentication
- Firewalls
- Access Control
- Secure Protocols (TLS)

**10. Scalability Plan**

- Cloud-based servers.
- Distributed processing.

- Load balancing.

**Question 10 – Threads and Multithreading Models**

**1. Threads Involved**

- Video Decoding
- Audio Synchronization
- Subtitle Generation
- Buffering
- Advertisement Insertion

**2. User-Level vs Kernel-Level Threads**

| User-Level Threads | Kernel-Level Threads |
|---|---|
| Managed by user library | Managed by OS |
| Faster creation | Better parallelism |
| One blocked thread blocks all | Other threads continue running |

**3. Multithreading Models**

| Model | Suitability |
|---|---|
| Many-to-One | Poor |
| One-to-One | Best |
| Many-to-Many | Good for large systems |

**4. Effect of Thread Blocking**

- Video freezes.
- Audio-video synchronization issues.
- Increased buffering.
- Poor user experience.

**5. Recommended Model**

**One-to-One Model**

Each user thread maps to one kernel thread, allowing true parallel execution on multicore processors.

**6. Tenfold Increase in Users**

- Higher CPU and memory usage.
- More network traffic.
- Increased thread count.
- Requires load balancing and server scaling.

**7. Improve Synchronization**

- Mutex Locks
- Semaphores
- Condition Variables
- Thread Pools

**8. Thread Management Strategy**

Main Thread



├── Video Thread

├── Audio Thread

├── Subtitle Thread

├── Buffer Thread

└── Advertisement Thread

A thread pool manages worker threads efficiently.

**9. Role of Multithreading**

- Executes tasks concurrently.
- Improves responsiveness.
- Maximizes CPU utilization.
- Enhances streaming performance.

**10. Monitoring Framework**

Thread Monitor

CPU Usage

↓

Memory Usage

↓

Thread Status

↓

Error Detection

↓

Performance Reports